

How the EXC LSTM Stock Predictor Works

From the basics of LSTMs to MC-Dropout forecasting

This document walks through every function in **main.py**, explaining the theory behind each decision in plain language. No deep learning background required.

1. What is an LSTM?

Recurrent Neural Networks (RNNs)

A standard neural network processes one input at a time and has no memory. A Recurrent Neural Network (RNN) adds a loop: the output at step t is fed back as extra input at step $t+1$. This gives the network a form of short-term memory — useful for sequences like time series, text, or audio.

Analogy: imagine reading a sentence word by word. Each word makes more sense when you remember what came before it. An RNN does the same with numbers.

The Vanishing Gradient Problem

When training an RNN on long sequences (e.g. 60 days of prices), gradients must travel backwards through every time step. Each step multiplies the gradient by a small weight, so after 60 steps the gradient shrinks to nearly zero. The network stops learning anything from events far in the past — this is the **vanishing gradient problem**.

How LSTMs Solve It

Long Short-Term Memory networks (LSTMs, Hochreiter & Schmidhuber, 1997) replace the simple RNN cell with a more complex unit containing two conveyor belts of information:

- **Cell state (C)** — the long-term memory. It flows through the sequence with only minor, targeted modifications.
- **Hidden state (h)** — the short-term memory / working output used at each step.

Three **gates** control what information is kept or discarded:

Gate	Question it answers	Effect
Forget gate	What old memory should I erase?	Multiplies cell state by values close to 0 (forget) or 1 (keep)

Input gate	What new information should I store?	Decides which new values are written into the cell state
Output gate	What should I output right now?	Filters the cell state to produce the current hidden state h

Analogy: think of the cell state as a whiteboard. The forget gate decides what to erase, the input gate writes new notes, and the output gate decides what you read aloud to the next step.

2. The Data Pipeline

load_and_prepare_data()

This function downloads 15 years of daily EXC (Exelon Corporation) stock data from Yahoo Finance and engineers seven features the LSTM will learn from:

Feature	What it measures	Why it helps
Close	Daily closing price (USD)	The target variable — what we want to predict
Volume	Shares traded that day	High volume often signals conviction behind a price move
SMA_20	Simple 20-day moving average	Smooths noise; shows the medium-term trend direction
EMA_20	Exponential 20-day moving average	Like SMA but weights recent days more heavily
RSI_14	Relative Strength Index (0-100)	Above 70 = overbought, below 30 = oversold; flags reversals
MACD	MACD difference (fast EMA - slow EMA)	Captures momentum shifts and trend changes
OBV	On-Balance Volume (cumulative)	Tracks whether volume is flowing into or out of the stock

MinMaxScaler — Normalisation

Raw values vary wildly: Close is ~\$40, Volume is in the millions, OBV can be in the billions. Neural networks learn poorly from such mismatched scales. **MinMaxScaler** rescales every feature to the range [0, 1]:

```
scaler = MinMaxScaler()
scaler.fit(values[:train_end]) # <-- only on training data
scaled = scaler.transform(values)
```

Why fit only on training data?

If you fit the scaler on the entire dataset (including future prices), the scaler's min/max would encode information about future prices into the training normalization. The model would unknowingly 'see' the future. This is called **data leakage** and produces metrics that look great but fail completely on real future data.

Fix: fit the scaler only on the training slice (70% of the data), then use those same statistics to transform the validation and test sets.

make_sequences() — The Sliding Window

LSTMs expect sequences, not individual rows. This function creates a **60-day rolling window**: for every day in the dataset it packages the previous 60 days as one input sample, with the next day's Close as the target.

```
# For i = 60, 61, 62 ...
X[i] = scaled[i-60 : i]    # shape: (60, 7) — 60 days x 7 features
y[i] = scaled[i, 0]        # column 0 = normalised Close
```

Why 60 days? It covers roughly three trading months — long enough to capture medium-term trends and indicator patterns without being so long that the network forgets the beginning of the window.

3. The Model: LSTMRegressor

The model class inherits from `nn.Module` (PyTorch's base class for all neural networks). It has three layers:

```
class LSTMRegressor(nn.Module):
    def __init__(self, input_dim, hidden_dim, num_layers, dropout):
        self.lstm = nn.LSTM(input_dim, hidden_dim, num_layers,
                             batch_first=True, dropout=dropout)
        self.drop = nn.Dropout(dropout)
        self.fc    = nn.Linear(hidden_dim, 1)

    def forward(self, x):
        out, _ = self.lstm(x)          # process all 60 time steps
        return self.fc(self.drop(out[:, -1, :])) # predict from last step
```

nn.LSTM — the core

- **input_dim = 7** — one value per feature at each time step.
- **hidden_dim = 64** — each LSTM cell outputs a 64-dimensional vector (its hidden state).
- **num_layers = 2** — two LSTM layers stacked. The second layer receives the hidden states of the first as its input, letting it learn higher-level patterns.
- **batch_first=True** — input shape is `(batch, 60, 7)` instead of `(60, batch, 7)`.

nn.Dropout — Regularisation

During training, dropout randomly sets 20% of neuron outputs to zero at each forward pass. This forces the network to learn redundant representations — no single neuron can be relied upon too heavily. The result is a model that generalises better to unseen data rather than memorising the training set.

Analogy: it's like studying for an exam while randomly covering parts of your notes. You end up understanding the material deeply rather than memorising specific sentences.

nn.Linear — The Output Head

After processing 60 time steps, we take only the **last hidden state** (`out[:, -1, :]` — shape: `(batch, 64)`). This vector summarises everything the LSTM learned from the 60-day window. The linear layer compresses it from 64 dimensions down to 1 — the predicted normalised closing price.

4. Training: the `train()` function

Loss Function — `MSELoss`

`nn.MSELoss` computes the **Mean Squared Error** between predictions and true values. Squaring the errors penalises large mistakes more heavily than small ones, pushing the model to avoid big mispredictions.

- $MSE = \text{mean}((\text{predicted} - \text{actual})^2)$

Optimiser — Adam

Adam (Adaptive Moment Estimation) adjusts the learning rate individually for each parameter based on the history of gradients. It combines momentum (memory of past gradients) with RMSprop (adaptive step sizes). In practice it converges faster and more reliably than plain SGD for most deep learning tasks.

Gradient Clipping

```
nn.utils.clip_grad_norm_(model.parameters(), 1.0)
```

During backpropagation through 60 time steps, gradients can occasionally explode — growing so large they corrupt the weights in a single step. Clipping caps the global gradient norm at 1.0, preventing this while leaving normal-sized gradients untouched.

Learning Rate Scheduler — `ReduceLROnPlateau`

If validation loss does not improve for `LR_PATIENCE=5` epochs in a row, the learning rate is halved. This lets the model take large steps early in training (fast progress) and smaller, more precise steps later (fine-tuning without overshooting).

Early Stopping

At every epoch we check whether validation loss improved. If it does, we save a copy of the weights (`best_weights`). If it fails to improve for `ES_PATIENCE=15` consecutive epochs, training stops and the best weights are restored. This prevents **overfitting** — training too long makes the model memorise the training data and perform worse on new data.

```
if val_loss < best_val:
    best_weights = {k: v.clone() for k, v in model.state_dict().items()}
    no_improve = 0
else:
    no_improve += 1
    if no_improve >= ES_PATIENCE:
```

```
break
```

```
model.load_state_dict(best_weights) # restore the best checkpoint
```

In our run: training stopped at epoch 45 with best validation loss at epoch 30. Without early stopping, the model would have overfit for another 55 epochs.

5. Evaluation: `evaluate()` and `_inverse_close()`

The Inverse Transform Problem

The model predicts normalised Close values (between 0 and 1). To get real USD prices we need to invert the scaler. But `scaler.inverse_transform()` expects all 7 features — it doesn't know how to invert just one column. `_inverse_close()` works around this by building a dummy matrix of zeros with the predicted values placed in column 0 (the Close column), inverting the whole matrix, then extracting just column 0.

```
def _inverse_close(scaled_1d, scaler, n_feat):
    buf = np.zeros((len(scaled_1d), n_feat)) # dummy matrix of zeros
    buf[:, 0] = scaled_1d                    # place Close in column 0
    return scaler.inverse_transform(buf)[:, 0] # invert & extract column 0
```

Metrics Explained

Metric	Formula	Plain English	Our result
RMSE	$\sqrt{\text{mean}((\text{pred}-\text{true})^2)}$	Typical error in USD. Penalises large errors heavily.	\$11.4
MAE	$\text{mean}(\text{pred}-\text{true})$	Average absolute error in USD. More robust to outliers.	\$0.18
MAPE	$\text{mean}(\text{pred}-\text{true} /\text{true}) \times 100$	Average % error. Intuitive: how wrong are we on average?	1.87%

A MAPE of 1.87% means the model's predictions are off by less than 2 cents for every dollar of stock price on average — strong performance for a single-asset LSTM.

6. Forecasting with Uncertainty: `mc_forecast()`

Why Standard Forecasts Have No Uncertainty

A normal neural network in inference mode (`model.eval()`) produces a single deterministic prediction. It gives you one number with no indication of how confident it is. For financial forecasting this is dangerous — a 30-day projection with no error bars is false precision.

MC-Dropout: Uncertainty for Free

MC-Dropout (Gal & Ghahramani, 2016) exploits a key insight: if you keep dropout **active at inference time** (by calling `model.train()` instead of `model.eval()`), every forward pass randomly deactivates different neurons, producing a **different prediction** each time. Running 100 such passes gives you a **distribution** over predictions:

- **Mean** of 100 runs = the forecast.
- **Std** of 100 runs = the uncertainty band (MC +/-1 sigma).

```
model.train()    # dropout stays ON during inference

runs = []
for _ in range(MC_SAMPLES):    # 100 stochastic forward passes
    preds = roll_forward_30_days(model, seq)
    runs.append(preds)

future_mean = np.array(runs).mean(axis=0)    # central forecast
future_std  = np.array(runs).std(axis=0)    # uncertainty per day
```

Analogy: instead of asking one person to predict tomorrow's weather, you ask 100 slightly different versions of the same forecaster. Their agreement gives confidence; their disagreement reveals uncertainty.

`_recompute_last_indicators()` — Live Feature Updates

A naive forecast would freeze the technical indicators (SMA, EMA, RSI, MACD) at their last known values and just shift the window. This is wrong — if the predicted price rises sharply, RSI should reflect that overbought condition. `_recompute_last_indicators()` solves this by rebuilding all indicators from the growing price history after each predicted step:

```
for step in range(FORECAST_DAYS):
    next_close = model.predict(current_window)    # predict next price

    close_buf.append(next_close)                  # extend price history
    sma, ema, rsi, macd = _recompute_last_indicators(close_buf, volume_buf)
    obv += volume if next_close > prev_close else -volume    # incremental OBV

    new_row = [next_close, volume, sma, ema, rsi, macd, obv]
    current_window = slide_window(new_row)        # shift window by 1 day
```

*OBV is updated **incrementally** rather than recomputing the full cumulative sum each step — this avoids an $O(n)$ operation inside the inner loop of $100 \times 30 = 3,000$ iterations.*

7. Results

Data & Training Setup

Ticker	EXC (Exelon Corporation)
--------	--------------------------

History	15 years of daily OHLCV data
Features	7 (Close, Volume, SMA20, EMA20, RSI14, MACD, OBV)
Split	70% train / 15% val / 15% test (no leakage)
Window	60 trading days
Model	2-layer LSTM, hidden=64, dropout=0.2 (52,033 params)
Optimiser	Adam lr=0.001, gradient clip=1.0
Scheduler	ReduceLROnPlateau, patience=5, factor=0.5
Early stopping	Patience=15 epochs -> stopped at epoch 45
Uncertainty	MC-Dropout, 100 samples x 30 days

Test Set Metrics

RMSE = \$1.14 | MAE = \$0.80 | MAPE = 1.87%

These results mean the model's test-set predictions deviate from real prices by about **1.87% on average**. For a stock trading around \$40, that is roughly an \$0.75 average error — well within a single day's typical price swing.

Important Caveats

- **Past performance does not guarantee future results.** Financial markets are non-stationary; regime changes (recessions, policy shifts) can invalidate historical patterns.
- The model predicts **price level**, not **direction**. A low MAPE does not mean the model correctly predicts whether the stock goes up or down.
- The uncertainty band (MC +/-1 sigma) grows with time as errors compound over 30 days — treat distant forecasts with more scepticism.

Generated from **main.py** — LSTM_Pytorch project. Model weights saved to **EXC_lstm.pt**.